DEFENSIVE PUBLICATION

HACI: Human-AI Collaborative Interchange

A Semantic Markdown Profile for Deterministic Role Attribution
in Human-AI Collaborative Authoring

Author: David Lee Wise | **Organisation:** Bridge-Burners LLC | **Contact:** TriPod / ROOT0
Date of Disclosure: June 26, 2026 | **Version:** 0.1 | **File Extension:** .haci

ABSTRACT

This disclosure describes HACI (Human-AI Collaborative Interchange), a semantic profile of Markdown that enables deterministic role attribution in documents co-authored by humans and machines. HACI introduces two new prefix symbols (! and ?) alongside one case convention to classify every line in a document by authorial role: human command, machine proposal, documentation, evidence, open question, or executable code. The dialect requires no new language, no NLP, and no language model to parse. It remains valid Markdown in any editor. A reference parser is provided. The specification self-hosts: it is written in its own format and parses correctly under its own rules.

1. BACKGROUND AND PROBLEM STATEMENT

Documents produced through human-AI collaboration currently carry no inline record of who authored which content. When a human directs an AI system to generate text, code, or analysis, the resulting artifact flattens all contributions into a single undifferentiated voice. This creates three problems:

Attribution loss. After a collaborative session, there is no way to determine which portions of a document represent human intent, which are machine-generated proposals, and which are verified evidence from external sources.

Authority ambiguity. A reader cannot distinguish a human directive (which carries organisational authority) from a machine suggestion (which is advisory) without external context that the document itself does not preserve.

Audit opacity. For compliance, governance, and intellectual-property purposes, the provenance of each claim in a document matters. Current collaborative workflows destroy this provenance at the point of composition.

Existing solutions (tracked changes, version control diffs, chat logs) preserve provenance externally but not within the document itself. HACI addresses this by embedding role attribution directly in the document's syntax, using conventions that are simultaneously human-readable and machine-parseable.

2. DESCRIPTION OF THE DISCLOSURE

2.1 Design Principles

HACI is governed by five normative principles: (1) human intent is explicit; (2) machine contributions are identifiable; (3) evidence is distinguishable from intent; (4) the dialect must

remain valid Markdown; and (5) the grammar must stay minimal.

2.2 The Symbol Table

HACI adds exactly two new prefix symbols to standard Markdown and applies one case convention. The complete grammar is as follows:

Symbol	Role	Rule	Deterministic
!	Human Command	Line prefix	Yes
?	Question (AI)	Line prefix + lowercase body	Yes
?! or ?+UC	Question (Human)	Line prefix + uppercase body	Yes
>	Evidence / Output	Line prefix (Markdown blockquote)	Yes
#	Heading	Line prefix (Markdown native)	Yes
`` or ~~~	Code Block	Fence markers (Markdown native)	Yes
A-Z first alpha	Documentation	First alphabetic character uppercase	Yes
a-z first alpha	AI Proposal	First alphabetic character lowercase	Yes

2.3 Lexer Rules

The parser operates as a single-pass, line-by-line lexer with no lookahead. Each line is classified independently according to the following priority order:

Priority	Rule	Pattern
1	Human Command	Line starts with ! (after optional whitespace)
2	Question	Line starts with ? (subtyped by case of body text)
3	Evidence	Line starts with > (maps to Markdown blockquote)
4	Code Fence	Line starts with ``` or ~~~ (toggle fence state)
5	Heading	Line starts with # followed by space
6	Case Convention	First alphabetic character: upper = Doc, lower = AI
7	Fallback	No alphabetic characters: treated as Documentation

2.4 Code Fence State Tracking

The parser maintains a boolean fence-state flag. When a line matches the code-fence pattern and the parser is not inside a fence, the flag is set and subsequent lines are accumulated as CODE content until a matching fence-close line is encountered. Role-tag patterns appearing inside a fenced block are treated as content, not as role markers. An unclosed fence at end-of-document is treated as a CODE block with a parser warning.

2.5 Question Subtyping

The ? prefix identifies a question. The case convention (Rule 6) is applied to the body text after the ? symbol to determine the asker. A lowercase body indicates an AI question (exploratory). An uppercase or sentence-case body indicates a human question (directive). The composite prefix ?! (or !?) explicitly marks a human question regardless of body case. This composes existing symbols without introducing new ones.

2.6 The Case Convention

For lines not matched by any prefix rule, the first alphabetic character determines the role. An uppercase first alpha classifies the line as Documentation; a lowercase first alpha classifies it as AI-authored content (proposal, reasoning, or implementation). This is a writing convention enforced by the author, not a heuristic inferred by the parser. Brand names or terms that begin with a lowercase letter (e.g., macOS, x86) are handled by rephrasing: the author writes "The macOS build requires..." rather than "macOS requires..." to ensure the first word carries the case signal. This is a two-second writing habit.

3. SELF-HOSTING VERIFICATION

The HACI v0.1 specification is written in its own format (file: haci-spec.haci). A reference parser (file: haci.py) processes the specification and produces the following role distribution, confirming that the dialect can describe its own parsing rules without ambiguity:

Role	Block Count	Character Count
HUMAN	5	214
AI	23	1,039
DOCUMENTATION	39	2,135
EVIDENCE	4	146
CODE	9	1,287
HEADING	19	366
AI_QUESTION	2	99
META	1	17
TOTAL	**102**	**5,303**

4. EXAMPLE DOCUMENT

The following is a complete HACI document demonstrating all role types in a realistic collaborative workflow:

```
! BUILD THE TASK SCHEDULER

The scheduler manages work distribution across threads.
Each worker pulls from a priority queue.

allocate thread pool based on cpu count
```

```
initialize priority heap
wire up the completion callbacks

? should we use a work-stealing strategy or round-robin?

> benchmarks show work-stealing is 2.3x faster for uneven loads

! USE WORK-STEALING

implement work-stealing with per-thread deques

```python
class Scheduler:
    def __init__(self, n_workers):
        self.deques = [deque() for _ in range(n_workers)]
```

> scheduler instantiated with 8 workers
```

In this example, the parser produces 15 blocks across 6 roles: 2 HUMAN commands, 4 AI proposals, 3 DOCUMENTATION lines, 2 EVIDENCE observations, 1 AI_QUESTION, and 1 CODE block. Every role is deterministically assigned by prefix or case convention with no ambiguity.

5. COMPATIBILITY AND FILE FORMAT

HACI is a profile of Markdown, not a replacement. Every HACI document is valid Markdown. An HACI-unaware renderer (GitHub, VS Code, any Markdown editor) displays a readable document with natural formatting. An HACI-aware renderer adds role attribution through colour, margin labels, or structural metadata.

The designated file extension is **.haci** (Human-AI Collaborative Interchange). The extension .haci has no significant prior use in text-format contexts; existing uses are limited to binary formats (PlayStation 3D model data, SWAT hydrological humidity input) with no overlap. HACI documents SHOULD begin with the HTML comment <!-- HACI v0.1 --> to disambiguate from other formats.

6. REFERENCE IMPLEMENTATION

A reference parser and HTML renderer (haci.py, approximately 150 lines of Python) accompanies this disclosure. The parser's core lexer is approximately 40 lines, uses one regular expression, and contains zero role-specific special cases. It operates in a single pass with O(n) complexity where n is the number of lines in the document. The parser supports four output modes: block listing, HTML rendering with role-attributed colours, role statistics, and self-hosting verification.

7. RELATIONSHIP TO PRIOR ART

HACI is distinguished from prior work in the following respects:

> **Markdown extensions** (MultiMarkdown, GitHub Flavored Markdown, CommonMark extensions): these add structural features (tables, footnotes, task lists) but do not address authorial role attribution. They extend what Markdown can represent, not who said it.

Chat and API role fields (OpenAI, Anthropic message roles: system/user/assistant): these attribute roles at the message level in API requests but do not persist into the resulting document. Role attribution stops at the API boundary.

Document annotation systems (tracked changes, Git blame, collaborative editing): these preserve provenance externally (in metadata, diffs, or overlays) but not in the document's own syntax. The document itself remains unattributed.

Semantic markup languages (XML, YAML front matter, structured data formats): these provide full semantic tagging but sacrifice the human readability and tool compatibility that Markdown provides. HACI achieves semantic attribution while remaining valid Markdown.

HACI's contribution is the combination of inline role attribution, deterministic parsing, Markdown compatibility, and minimal syntax (two new symbols plus one case convention). No prior system known to the author achieves all four properties simultaneously.

8. SCOPE OF DISCLOSURE

This publication is a defensive disclosure intended to establish prior art and prevent the patenting of the following concepts by any party:

(a) The use of line-prefix symbols (specifically ! for human authority and ? for questions) combined with a first-alphabetic-character case convention to classify lines in a Markdown document by authorial role in human-AI collaborative workflows.

(b) The composition of prefix symbols (specifically ?! and !?) to create compound role classifications without introducing additional syntax.

(c) A single-pass, line-by-line lexer with code-fence state tracking that deterministically classifies every line of a Markdown-compatible document into one of a fixed set of authorial roles (human command, AI proposal, documentation, evidence, question, code, heading).

(d) A self-hosting specification format in which the specification document is written in the dialect it defines and is parseable by the reference implementation it describes.

(e) The file extension .haci as an identifier for Human-AI Collaborative Interchange documents.

David Lee Wise / Bridge-Burners LLC / 2026-06-26